

Exhaustive Report: Numerical Stability of Explicit and Implicit Methods for a Bounded Memristor ODE

John Louis Lagramada

Patrick Villareal

Contents

1	Overview	3
1.1	Purpose of this report	3
1.2	Main idea in one paragraph	3
1.3	What the reader should look for	3
2	Background Concepts	3
2.1	What is a resistor?	3
2.2	What is a memristor?	4
2.3	Why the state variable matters	4
2.4	What is a hysteresis loop?	4
2.5	Why hysteresis is useful in this project	5
3	Mathematical Model	5
3.1	State equation	5
3.2	Meaning of each parameter	5
3.3	Why this is an initial value problem	5
3.4	Why numerical methods are needed	6
3.5	Physical bounds	6
4	Numerical Methods	6
4.1	Time discretization	6
4.2	Explicit Euler	6
4.3	Fourth-order Runge-Kutta	7
4.4	Implicit Euler	7
4.5	Important limitation of implicit methods	7
4.6	Raw and clipped trajectories	7
5	Experiment Design	7
5.1	Simulation interval	7
5.2	Step sizes	8
5.3	Why compare different step sizes?	8
5.4	Runtime measurement	8

6	Results and Plot Explanations	8
6.1	How to read the state trajectory plot	8
6.2	Presentation explanation for Figure 1	10
6.3	Interpretation of the state trajectories	10
6.4	How to read the boundary violation plot	10
6.5	Presentation explanation for Figure 2	10
6.6	Interpretation of the boundary violations	11
6.7	How to read the V-I hysteresis plot	11
6.8	Why clipping is used for V-I curves	11
6.9	Presentation explanation for Figure 3	11
6.10	Interpretation of the hysteresis loops	12
6.11	How to read the runtime plot	12
6.12	Presentation explanation for Figure 4	12
7	Method-by-Method Discussion	12
7.1	Explicit Euler	12
7.2	RK4	13
7.3	Implicit Euler	13
8	Important Distinctions	13
8.1	Accuracy versus stability	13
8.2	Numerical validity versus physical validity	13
8.3	Raw result versus clipped result	13
9	Limitations	13
9.1	Simplified memristor model	13
9.2	No adaptive step size	14
9.3	No built-in bound-preserving solver	14
9.4	Runtime is environment-dependent	14
10	Conclusion	14
10.1	Main conclusion	14
10.2	What the plots collectively show	14
10.3	Final presentation takeaway	14

1 Overview

1.1 Purpose of this report

This report explains the project from the ground up. It is written for a reader who may not know what a memristor is, what a hysteresis loop means, or why numerical methods are needed for ordinary differential equations. The goal is to make the simulation, the formulas, the plots, and the conclusions understandable enough for a presentation.

The project compares three numerical methods:

- Explicit Euler,
- fourth-order Runge-Kutta, usually called RK4,
- Implicit Euler.

These methods are used to approximate the time evolution of a memristor's internal state. The important question is not only whether the methods produce numbers, but whether those numbers stay physically meaningful.

1.2 Main idea in one paragraph

A memristor is an electrical device whose resistance depends on its past behavior. In this model, the device has an internal state $w(t)$, representing how much of the device is in a low-resistance doped state. Since this state is a physical length, it must stay between 0 and the full device thickness D . The project applies sinusoidal voltage to the device and numerically solves the resulting initial value problem. The numerical methods are compared by checking their state trajectories, their boundary violations, their voltage-current hysteresis loops, and their runtime cost.

1.3 What the reader should look for

There are four main outputs:

- The state trajectory plot shows how $w(t)/D$ changes over time.
- The boundary violation plot shows whether a numerical method leaves the physical interval $0 \leq w \leq D$.
- The V-I hysteresis plot shows how current responds to voltage over repeated sinusoidal cycles.
- The runtime plot shows how much computational work each method costs per step.

2 Background Concepts

2.1 What is a resistor?

A resistor is a basic electrical component that opposes current. For an ideal linear resistor, Ohm's law says

$$v = iR,$$

where v is voltage, i is current, and R is resistance. If R is constant, the current is simply

$$i = \frac{v}{R}.$$

For a fixed resistor, the device has no memory. If the same voltage is applied at two different times, the same current is expected.

2.2 What is a memristor?

A memristor is different because its effective resistance depends on its internal state. The word “memristor” comes from memory and resistor. A memristor can be thought of as a resistor with memory: its current response depends not only on the present voltage, but also on how the device arrived at its present internal state.

In the HP TiO₂ model used here, the device is treated as a thin film with two regions:

- a doped region with low resistance,
- an undoped region with high resistance.

The internal variable $w(t)$ is the width of the doped region. The full device thickness is D . Therefore:

$$0 \leq w(t) \leq D.$$

If $w(t) = 0$, none of the device is in the doped low-resistance state. If $w(t) = D$, the whole device is doped. Values below 0 or above D are physically impossible.

2.3 Why the state variable matters

The memristor resistance changes as $w(t)$ changes. The model defines the memristance as

$$M(w) = R_{\text{on}} \frac{w}{D} + R_{\text{off}} \left(1 - \frac{w}{D}\right).$$

Here:

- R_{on} is the resistance when the device is fully doped,
- R_{off} is the resistance when the device is undoped,
- w/D is the fraction of the device that is doped.

This is a weighted average. When w/D is large, the device behaves more like the low-resistance state. When w/D is small, it behaves more like the high-resistance state.

2.4 What is a hysteresis loop?

A hysteresis loop appears when a system’s output depends on its history. In this project, the input is voltage and the output is current. If the device had no memory and behaved like a fixed resistor, the voltage-current graph would be a single straight line. A memristor can produce a loop instead, because the current depends on the changing internal state $w(t)$.

The project uses a sinusoidal voltage:

$$v(t) = V_0 \sin(\omega t).$$

As time advances, the voltage rises, falls, becomes negative, and returns. If current is plotted against voltage over this cycle, the curve can trace a loop. The loop is called pinched because an ideal memristor’s V-I curve passes through the origin: when voltage is zero, current is also zero.

2.5 Why hysteresis is useful in this project

The hysteresis loop is a visual test of whether the simulation is physically meaningful. If the internal state is computed poorly, the current can become distorted because current is calculated from

$$i(t, w) = \frac{v(t)}{M(w)}.$$

If w leaves the physical interval, then the memristance is no longer physically valid. For this reason, the project stores two versions of the state:

- the raw numerical state, used to detect numerical overshoot,
- the clipped physical state, used to compute V-I plots.

3 Mathematical Model

3.1 State equation

The memristor state equation used in this project is

$$\frac{dw}{dt} = \frac{\mu_v R_{\text{on}} v(t)}{D^2 M(w)}.$$

The right-hand side tells us how fast $w(t)$ changes at time t . This change depends on the applied voltage $v(t)$ and the current memristance $M(w)$.

3.2 Meaning of each parameter

Symbol	Meaning	Default value used
R_{on}	fully doped resistance	$100 \, \Omega$
R_{off}	undoped resistance	$16\,000 \, \Omega$
D	full device thickness	$10 \times 10^{-9} \, \text{m}$
μ_v	ion mobility	$1 \times 10^{-14} \, \text{m}^2 \, \text{V}^{-1} \, \text{s}^{-1}$
V_0	voltage amplitude	$1 \, \text{V}$
f	drive frequency	$1 \times 10^8 \, \text{Hz}$
$w(0)$	initial state	$0.5D$

Table 1: Default simulation parameters.

3.3 Why this is an initial value problem

An ordinary differential equation gives a rule for the derivative of an unknown function. Here, the unknown function is $w(t)$. The equation gives dw/dt , not $w(t)$ directly. To obtain a specific trajectory, an initial value is needed:

$$w(0) = 0.5D.$$

Together, the derivative equation and the initial value form an initial value problem, or IVP.

3.4 Why numerical methods are needed

Some differential equations can be solved exactly with formulas. Many nonlinear equations cannot be solved conveniently in closed form. The memristor equation is nonlinear because $M(w)$ depends on w , and w itself is the unknown state. Instead of deriving a closed-form solution, the project approximates the solution step by step using numerical methods.

3.5 Physical bounds

The most important physical rule is

$$0 \leq w(t) \leq D.$$

In normalized form, this is

$$0 \leq \frac{w(t)}{D} \leq 1.$$

The plots use w/D because it is easier to interpret. A value of 0.5 means the doped region is half the device thickness. A value above 1 means the numerical method has produced an impossible state.

4 Numerical Methods

4.1 Time discretization

Numerical methods replace continuous time with discrete time points:

$$t_0, t_1, t_2, \dots, t_n.$$

The distance between two neighboring points is the step size:

$$h = t_{k+1} - t_k.$$

At each step, the method estimates the next state w_{k+1} from the current state w_k .

4.2 Explicit Euler

Explicit Euler is the simplest method in the project. It uses the slope at the current point:

$$w_{k+1} = w_k + hf(t_k, w_k).$$

The method is easy to understand: it moves forward along the current tangent line. It is cheap because it evaluates the derivative only once per step.

The weakness is that it trusts the current slope too much. If the step size is large, the tangent-line approximation can overshoot. In a bounded physical model, this can push w outside the interval $[0, D]$.

4.3 Fourth-order Runge-Kutta

RK4 improves on Euler by sampling several slopes inside one step. The classical formula is

$$\begin{aligned}k_1 &= f(t_k, w_k), \\k_2 &= f(t_k + h/2, w_k + hk_1/2), \\k_3 &= f(t_k + h/2, w_k + hk_2/2), \\k_4 &= f(t_k + h, w_k + hk_3), \\w_{k+1} &= w_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).\end{aligned}$$

RK4 is more accurate than Explicit Euler for smooth solutions, but it is still explicit. This means it still predicts the future from sampled slopes and does not automatically enforce physical bounds.

4.4 Implicit Euler

Implicit Euler uses the slope at the future point:

$$w_{k+1} = w_k + hf(t_{k+1}, w_{k+1}).$$

This is more difficult because w_{k+1} appears on both sides. The method must solve a nonlinear equation:

$$F(w_{k+1}) = w_{k+1} - w_k - hf(t_{k+1}, w_{k+1}) = 0.$$

The notebook solves this using Newton iteration. Near difficult raw states, it falls back to the equivalent scalar quadratic form of the same update. This keeps the implementation robust while still preserving the implicit method's intended equation.

4.5 Important limitation of implicit methods

Implicit methods often have better stability properties than explicit methods for many equations. However, implicit does not mean physically bounded. The method can still produce $w < 0$ or $w > D$ unless the model or algorithm explicitly enforces those constraints.

4.6 Raw and clipped trajectories

The project intentionally stores raw numerical outputs. This is necessary because clipping every update would hide overshoot. If a method produces $w = 1.4D$, that is important evidence about the method's behavior.

For V-I plots, a clipped state is also computed:

$$w_{\text{clipped}} = \min(\max(w, 0), D).$$

This clipped version is used only when a physically valid memristance is required for visualization.

5 Experiment Design

5.1 Simulation interval

The voltage is sinusoidal, so it repeats every period:

$$T = \frac{1}{f}.$$

The notebook simulates multiple periods so that the repeated memory behavior can be observed. The x-axis in the state plot is normalized as time divided by period. This lets the reader see how many voltage cycles have elapsed.

5.2 Step sizes

The notebook tests several step sizes:

$$T/400, \quad T/80, \quad T/20, \quad T/8.$$

Here, $T/400$ means 400 time steps per voltage period. This is a fine step. $T/8$ means only 8 time steps per voltage period. This is deliberately coarse.

5.3 Why compare different step sizes?

The same method can behave well with a small step size and poorly with a large step size. Comparing step sizes shows whether a method is sensitive to discretization. This is especially important in physical simulations, where a visually smooth curve can still be wrong if it violates physical limits.

5.4 Runtime measurement

Runtime is measured using `time.perf_counter()` over repeated solver calls. The report uses median runtime to reduce one-off timing noise. Runtime is not the main scientific result, because small notebook simulations are affected by Python overhead and machine conditions. Still, it is useful for showing relative computational cost.

6 Results and Plot Explanations

6.1 How to read the state trajectory plot

Figure 1 shows $w(t)/D$, the normalized internal state, over time. The horizontal axis is time divided by the voltage period. The vertical axis is the fraction of the device that is doped.

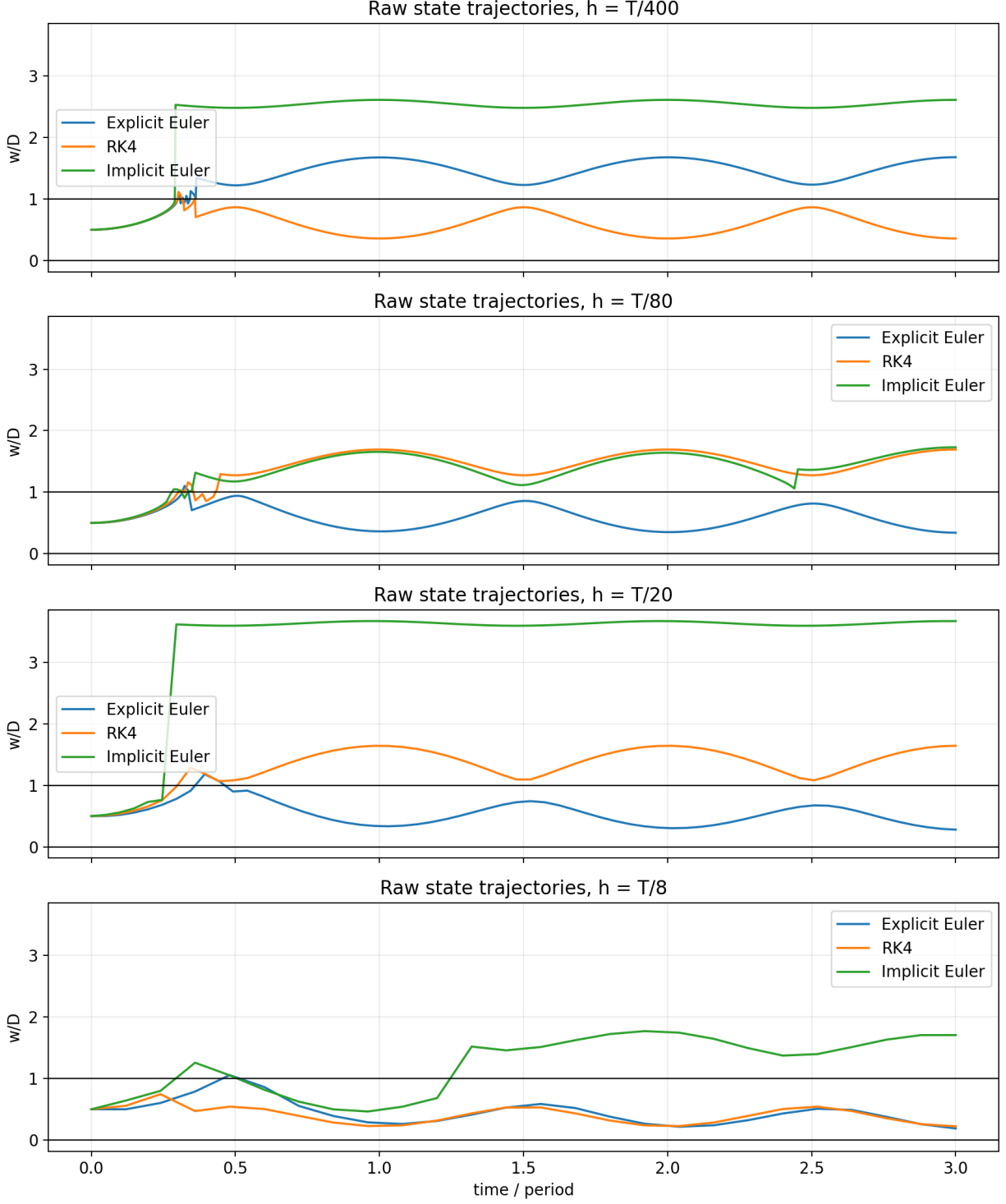


Figure 1: Raw normalized state trajectories for all methods and step sizes.

The two most important reference levels are:

$$w/D = 0, \quad w/D = 1.$$

The physically valid region is between these two lines. Any curve above 1 or below 0 is a raw

numerical result that violates the physical state constraint.

6.2 Presentation explanation for Figure 1

When presenting this figure, explain it as follows:

The plot shows the internal state of the memristor. A value of 0 means the doped region has zero width, and a value of 1 means it fills the device. The physical solution should stay between 0 and 1. We deliberately keep raw numerical values so we can see when a method overshoots. The different panels show different time steps. A fine step has many points per voltage period, while a coarse step has fewer points. The more a curve leaves the interval, the less physically meaningful that raw numerical trajectory is.

6.3 Interpretation of the state trajectories

The state trajectories show that step size strongly affects the raw solution. Even higher-order or implicit methods can leave the physical interval in this stress-case setup. This is not a contradiction. RK4 improves local accuracy for smooth problems, and Implicit Euler can improve stability for certain modes, but neither method automatically knows that w is a physical length unless the algorithm includes bound enforcement.

6.4 How to read the boundary violation plot

Figure 2 compresses the overshoot information into a bar chart. The vertical axis is the maximum violation normalized by D . A value of zero means the raw trajectory stayed within the physical interval for that method and step size.

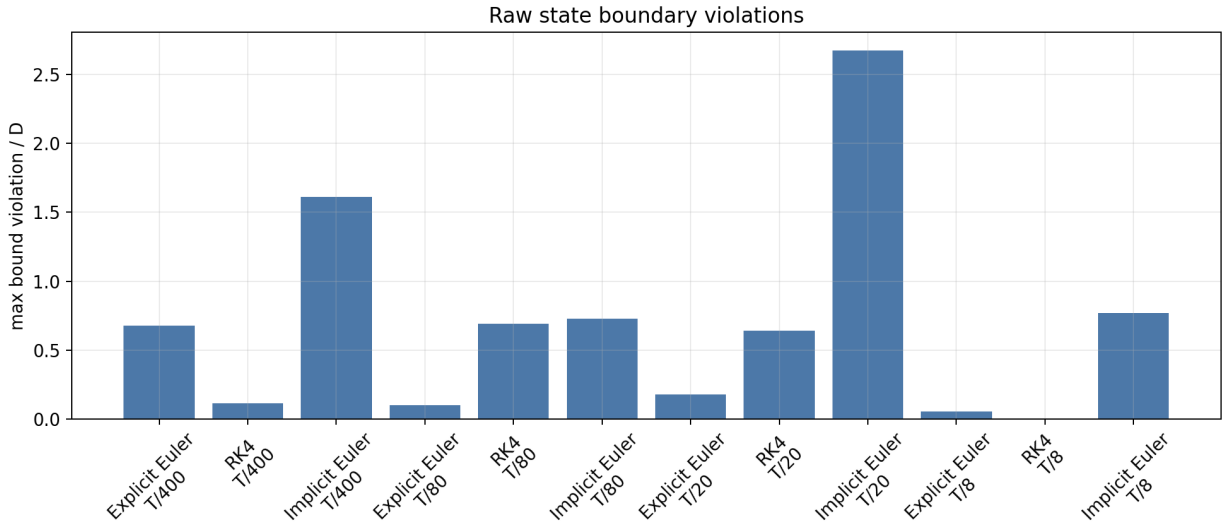


Figure 2: Maximum raw state boundary violation normalized by device thickness.

6.5 Presentation explanation for Figure 2

For presentation, this figure should be described as the direct physical-validity check:

This bar chart answers a simple question: how far did the raw computed state leave the physically allowed interval? A zero-height bar means no overshoot was observed. A taller bar means the numerical state became more physically unrealistic. This plot is important because it separates numerical smoothness from physical validity.

6.6 Interpretation of the boundary violations

The boundary plot confirms that the methods do not behave identically. Explicit Euler is simple and step-size dependent. RK4 can reduce error when the solution remains smooth and sufficiently resolved, but it can still overshoot under stress. Implicit Euler solves a future-time equation, but it is still solving the unbounded mathematical update; therefore, it can also leave the physical interval.

6.7 How to read the V-I hysteresis plot

Figure 3 plots current against voltage. Voltage is on the horizontal axis. Current is on the vertical axis. Because the voltage is sinusoidal, the system repeatedly moves forward and backward through voltage values.

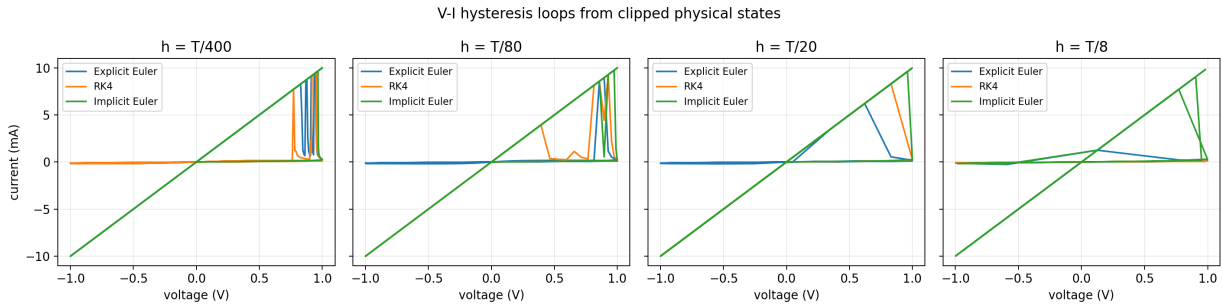


Figure 3: Voltage-current hysteresis loops computed from clipped physical states.

If the device were a fixed resistor, the V-I curve would be a straight line. A loop indicates memory: the current at a given voltage can differ depending on whether the voltage is increasing or decreasing and depending on the internal state history.

6.8 Why clipping is used for V-I curves

The hysteresis plot uses clipped states because current is computed from memristance, and memristance is only physically meaningful when w is within the device. This does not erase the overshoot analysis, because overshoot is already measured using raw states in the previous figures and tables. The purpose of clipping here is to show the physically interpretable V-I response.

6.9 Presentation explanation for Figure 3

For presentation, use this explanation:

The V-I plot shows the device's memory effect. Instead of getting a single straight line like an ordinary resistor, we get loops because the resistance changes with the internal state. The loops are computed from clipped states, meaning we force the state into the physical interval only for this visualization. The raw overshoot is still shown separately in the state and boundary plots.

6.10 Interpretation of the hysteresis loops

The V-I loops show how numerical differences in $w(t)$ influence the observable electrical behavior. Methods and step sizes that produce different clipped state histories can produce different loop shapes. This is important because in experiments and applications, the current-voltage relationship is often the visible output, while the internal state is hidden.

6.11 How to read the runtime plot

Figure 4 shows median runtime per time step. The y-axis is measured in microseconds per step. This normalizes the cost so that methods with different numbers of total steps can be compared fairly.

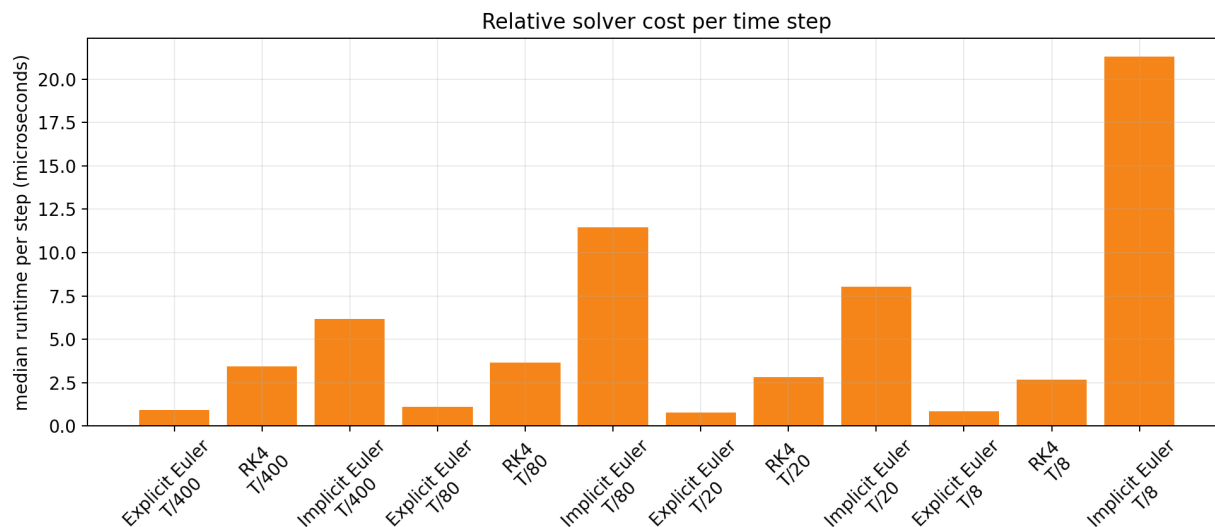


Figure 4: Median runtime per step over repeated solver calls.

6.12 Presentation explanation for Figure 4

For presentation, explain runtime as a secondary metric:

This plot compares computational cost per step. Explicit Euler is cheapest because it evaluates the derivative once. RK4 is more expensive because it evaluates four slopes per step. Implicit Euler is usually more expensive because it solves a nonlinear equation at each step. We should not overinterpret the exact numbers, but the relative cost pattern is useful.

7 Method-by-Method Discussion

7.1 Explicit Euler

Explicit Euler is useful as a baseline because it is simple and transparent. It has low computational cost, but it is only first-order accurate. It is also sensitive to step size. In this project, that sensitivity matters because the state is physically bounded.

The main lesson from Explicit Euler is that a method can be easy to implement but fragile under coarse time discretization.

7.2 RK4

RK4 is a higher-order explicit method. It is generally much more accurate than Explicit Euler when the step size is sufficiently small and the trajectory is smooth. However, RK4 still does not enforce $0 \leq w \leq D$. It can reduce numerical error, but it does not automatically solve the physical boundary problem.

The main lesson from RK4 is that higher accuracy is not the same as physical constraint preservation.

7.3 Implicit Euler

Implicit Euler uses future-time information. This often improves stability for certain decaying or stiff problems. In this project, however, the method still solves an unbounded equation unless additional constraint handling is added. The raw implicit trajectory can therefore overshoot.

The main lesson from Implicit Euler is that implicitness can help with stability, but it does not replace physical modeling decisions.

8 Important Distinctions

8.1 Accuracy versus stability

Accuracy asks whether a method is close to the true solution. Stability asks whether errors grow uncontrollably. A method can be accurate for small steps but unstable or physically invalid for large steps. The project compares both behavior and physical validity.

8.2 Numerical validity versus physical validity

A numerical method can produce a finite number without producing a physically meaningful state. In this project, any raw state outside $[0, D]$ is physically invalid. This is why the boundary violation plot is essential.

8.3 Raw result versus clipped result

The raw result tells us what the method actually computed. The clipped result tells us what current would look like if the state were forced back into the physical range. Both are useful, but they answer different questions.

9 Limitations

9.1 Simplified memristor model

The model used here is the linear ion drift HP TiO₂ model. Real memristors can have more complicated nonlinear drift, threshold effects, window functions, temperature effects, and device variability. This project focuses on a simple model because the goal is numerical-method comparison.

9.2 No adaptive step size

The notebook uses fixed step sizes. Adaptive solvers can change step size automatically based on error estimates. They are not used here because the project aims to implement and compare the course-level methods directly.

9.3 No built-in bound-preserving solver

The project detects and measures boundary violations, but the raw methods are not modified to enforce bounds during each update. This is intentional because the proposal asks to compare overshoot behavior. A future extension could add bound-preserving variants.

9.4 Runtime is environment-dependent

Runtime measurements depend on hardware, Python overhead, and system load. They are useful for relative comparison within this notebook but should not be treated as universal benchmark values.

10 Conclusion

10.1 Main conclusion

The project shows that solving a memristor ODE is not only about producing a curve. The computed internal state must also remain physically meaningful. Explicit Euler, RK4, and Implicit Euler differ in accuracy, cost, and stability behavior, but none of them automatically enforces the physical bound $0 \leq w \leq D$.

10.2 What the plots collectively show

The state trajectory plot shows the raw evolution of the internal state. The boundary violation plot directly identifies unphysical overshoot. The V-I hysteresis plot shows how the state affects observable electrical behavior. The runtime plot shows the computational cost tradeoff. Together, the figures support the central message: method choice, step size, and boundary handling all matter.

10.3 Final presentation takeaway

For a presentation, the clean takeaway is:

The memristor has memory because its resistance depends on an internal state. Numerically solving that state equation requires care. A method can look reasonable but still produce impossible internal states. Therefore, when simulating bounded physical systems, we must compare not only accuracy and runtime, but also whether the numerical method respects physical constraints.